PROJECT REPORT

PEMROGRAMAN ROBOTIKA DENGAN KIT iMCLab

Integrasi ESP32, Kontrol Motor DC (PWM), Kendali Python, IoT Blynk, dan Over-The-Air (OTA) Update

Disusun Oleh:

Ayala Septama Rahanda	23081010071
Rizky Ananda Ramadhan	23081010088
Narendra Putra Arianto	23081010113
Muhammad Faris Syaifulloh	23081010311

FAKULTAS ILMU KOMPUTER

PROGRAM STUDI INFORMATIKA

UNIVERSITAS PEMBANGUNAN NASIONAL "VETERAN" JAWA TIMUR

TAHUN 2026

ABSTRAK

Laporan proyek ini merangkum hasil pembelajaran dan penguasaan kompetensi dasar robotika dan sistem tertanam (embedded system) menggunakan kit iMCLab yang berbasis mikrokontroler ESP32. Pembelajaran dibagi menjadi empat tahap yang saling berkesinambungan, yaitu: (1) pemrograman dasar ESP32 dengan komunikasi serial dan pengaturan kecepatan motor DC menggunakan Pulse Width Modulation (PWM), (2) pengendalian kit dari komputer menggunakan bahasa pemrograman Python melalui pustaka pySerial lengkap dengan deteksi port otomatis, (3) pemantauan dan pengendalian jarak jauh berbasis Internet of Things (IoT) menggunakan platform Blynk, dan (4) pembaruan firmware secara nirkabel melalui mekanisme Over-The-Air (OTA).

Hasil pengujian menunjukkan bahwa seluruh tahap berjalan sesuai tujuan: motor DC dapat dinyalakan, dimatikan, dan diatur kecepatannya melalui Serial Monitor, program Python, maupun aplikasi Blynk; kecepatan putar (RPM) dapat dibaca menggunakan sensor optik dan ditampilkan secara waktu nyata; serta firmware dapat diperbarui lewat jaringan Wi-Fi tanpa kabel USB. Proyek ini memberi pemahaman menyeluruh mengenai alur dari perangkat keras, perangkat lunak, hingga konektivitas jaringan pada sebuah sistem kendali berbasis IoT.

Kata kunci: *ESP32, iMCLab, PWM, motor DC, pySerial, Blynk, IoT, OTA.*

BAB I PENDAHULUAN

1.1 Latar Belakang

Robotika dan sistem kendali modern tidak lagi berdiri sendiri. Sebuah aktuator seperti motor DC kini umumnya dikendalikan oleh mikrokontroler, dikontrol melalui komputer, dan dipantau dari jarak jauh melalui jaringan internet. Untuk memahami rantai tersebut secara utuh, diperlukan praktik langsung pada perangkat yang mengintegrasikan seluruh elemennya.

Kit iMCLab (Internet-based Motor Control Lab) menyediakan platform praktik tersebut. Kit ini memadukan mikrokontroler ESP32, driver motor, motor DC dengan baling-baling, LED indikator, serta sensor kecepatan putar. Melalui kit ini, mahasiswa dapat mempelajari pemrograman mikrokontroler, kendali kecepatan dengan PWM, komunikasi serial dengan komputer, hingga integrasi IoT dan pembaruan firmware nirkabel.

1.2 Tujuan

1. Memahami dasar komunikasi serial pada ESP32 dan pengaturan kecepatan motor DC dengan PWM.
2. Mampu mengendalikan kit iMCLab dari komputer menggunakan program Python.
3. Mampu mengintegrasikan kit iMCLab dengan platform IoT Blynk untuk kendali dan pemantauan jarak jauh.
4. Mampu melakukan pembaruan firmware secara nirkabel menggunakan OTA.
5. Mendokumentasikan proses, hasil, kendala, dan pelajaran yang diperoleh dalam bentuk laporan proyek.

1.3 Ruang Lingkup

Laporan ini mencakup empat modul praktik: Modul 1 (pemrograman dasar dan PWM), Modul 2 (kendali melalui Python), Modul 3 (integrasi Blynk), dan Modul 4 (OTA). Pembahasan difokuskan pada kendali satu motor DC dengan satu arah putaran serta pembacaan kecepatan melalui sensor optik.

BAB II LANDASAN TEORI

2.1 Mikrokontroler ESP32

ESP32 adalah mikrokontroler 32-bit dengan prosesor dual-core yang dilengkapi Wi-Fi dan Bluetooth bawaan. Pada praktik ini digunakan papan DOIT ESP32 DEVKIT V1 yang diprogram melalui Arduino IDE. Komunikasi dengan komputer dilakukan melalui chip jembatan USB-to-UART (misalnya Silicon Labs CP210x) sehingga papan muncul sebagai port serial (COM) di komputer.

2.2 Komunikasi Serial

Komunikasi serial (UART) mengirimkan data bit demi bit secara berurutan antara dua perangkat. Kedua sisi harus menggunakan baud rate yang sama; pada proyek ini digunakan 115200 baud. Di sisi ESP32, data dibaca melalui Serial.available() dan Serial.read(), sedangkan di sisi komputer dapat digunakan Serial Monitor Arduino IDE atau pustaka pySerial pada Python.

2.3 Pulse Width Modulation (PWM)

PWM mengatur daya rata-rata yang diterima beban dengan cara mengubah lebar pulsa (duty cycle) pada frekuensi tetap. Pada resolusi 8 bit, nilai duty cycle berada pada rentang 0 sampai 255; nilai 0 berarti motor mati dan 255 berarti daya penuh. Semakin besar duty cycle, semakin besar tegangan rata-rata pada motor sehingga putarannya semakin cepat. ESP32 menyediakan modul LEDC untuk membangkitkan PWM, yang pada program ini diatur dengan frekuensi 30 kHz.

2.4 Driver Motor dan Pembacaan RPM

Motor DC tidak dapat dihubungkan langsung ke pin ESP32 karena kebutuhan arusnya melebihi kemampuan pin. Karena itu digunakan driver motor (jembatan H) yang dikendalikan oleh dua pin arah dan satu pin enable yang menerima sinyal PWM. Kecepatan putar diukur dengan sensor optik dan piringan berlubang. Setiap lubang menghasilkan pulsa yang dihitung melalui interrupt (ISR), kemudian dikonversi menjadi putaran per menit dengan persamaan:

$$RPM = (\text{jumlah pulsa} / \text{jumlah lubang}) \times 60 \quad (\text{dihitung setiap 1 detik})$$

2.5 Internet of Things dan Blynk

IoT menghubungkan perangkat fisik ke jaringan internet sehingga dapat dipantau dan dikendalikan dari jarak jauh. Blynk adalah platform IoT yang menyediakan dashboard di web dan aplikasi ponsel. Komunikasi antara dashboard dan perangkat dilakukan melalui virtual pin

(V0, V1, dan seterusnya). Perangkat mengirim data dengan Blynk.virtualWrite() dan menerima perintah melalui fungsi callback BLYNK_WRITE().

2.6 Over-The-Air (OTA) Update

OTA memungkinkan firmware diunggah ke ESP32 melalui jaringan Wi-Fi tanpa kabel USB. Pustaka ArduinoOTA membuat ESP32 mengiklankan dirinya sebagai network port di jaringan lokal, sehingga muncul pada menu Port Arduino IDE. Fitur ini sangat berguna ketika perangkat sulit dijangkau secara fisik atau sudah terpasang pada sistem.

BAB III ALAT, BAHAN, DAN KONFIGURASI

3.1 Perangkat Keras

6. Kit iMCLab (ESP32 DOIT DEVKIT V1, driver motor, motor DC dengan baling-baling, sensor RPM, LED indikator).
7. Kabel USB untuk pemrograman dan komunikasi serial.
8. Adaptor daya untuk menyuplai kit tanpa kabel USB (digunakan pada pengujian OTA).
9. Laptop/PC, jaringan Wi-Fi, dan ponsel pintar.

3.2 Perangkat Lunak

10. Arduino IDE (board DOIT ESP32 DEVKIT V1) dengan pustaka WiFi, BlynkSimpleEsp32, dan ArduinoOTA.
11. Python 3.11 dengan pustaka pySerial, dijalankan pada Jupyter Notebook.
12. Platform Blynk (web dashboard dan aplikasi mobile).

3.3 Konfigurasi Pin dan Parameter

Tabel 1. Konfigurasi pin dan parameter pada program

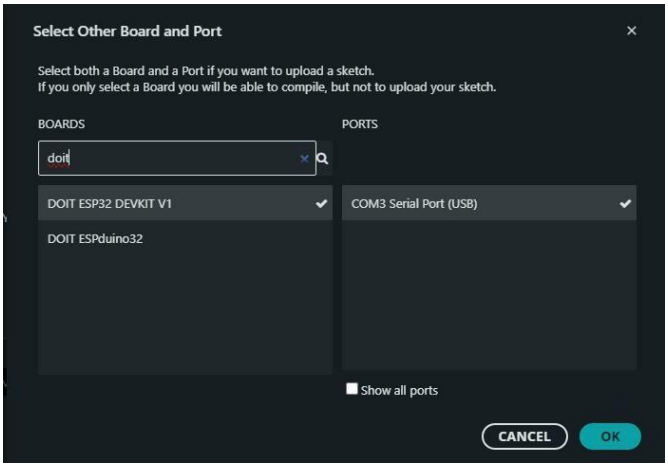
Komponen	Pin / Nilai	Keterangan
Motor – arah 1	GPIO 27	Output digital (HIGH) untuk menentukan arah putar
Motor – arah 2	GPIO 26	Output digital (LOW)
Motor – enable	GPIO 12	Menerima sinyal PWM dari kanal LEDC 0
LED indikator	GPIO 2	Menyala ketika duty cycle lebih dari 0
Sensor RPM	GPIO 13	Input dengan pull-up, interrupt pada sisi FALLING
Frekuensi PWM	30.000 Hz	Resolusi 8 bit (duty 0 sampai 255)
Jumlah lubang piringan	2	Dipakai pada rumus konversi pulsa ke RPM
Debounce sensor	500 µs	Menyaring pulsa palsu pada ISR
Baud rate serial	115200	Sama pada ESP32 dan program Python

BAB IV PEMBAHASAN DAN IMPLEMENTASI

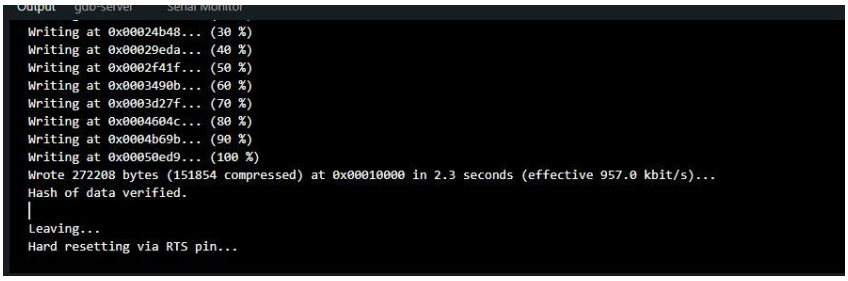
4.1 Modul 1: Pemrograman Dasar Robotika dengan Kit iMCLab

4.1.1 Persiapan dan unggah program

Langkah awal adalah menyiapkan Arduino IDE: memilih board DOIT ESP32 DEVKIT V1 dan port yang sesuai (COM3 pada komputer yang digunakan), kemudian mengunggah program dasar. Proses unggah dinyatakan berhasil ketika terminal menampilkan penulisan memori mencapai 100% dan ESP32 melakukan reset otomatis.



Gambar 1. Jendela Select Other Board and Port untuk memilih board DOIT ESP32 DEVKIT V1 dan port COM3.



Gambar 2. Log terminal Arduino IDE yang memperlihatkan proses unggah program selesai 100%.

4.1.2 Cara kerja program kendali serial dan PWM

Program imclab_naik_turun.ino menerima perintah teks melalui Serial Monitor. Setiap karakter yang masuk disimpan dalam buffer hingga ditemukan karakter baris baru, lalu buffer dianalisis. Perintah yang didukung ditunjukkan pada Tabel 2.

Tabel 2. Perintah serial pada program kontrol motor

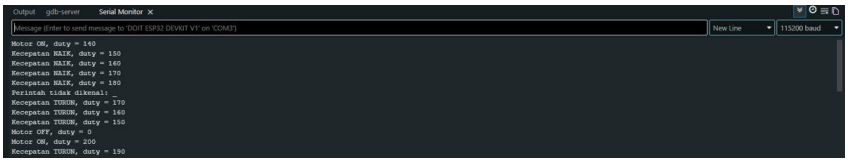
Perintah	Fungsi
1	Motor ON; jika duty cycle masih 0, dipakai nilai default
0	Motor OFF (duty cycle = 0) dan LED padam

Perintah	Fungsi
+	Menaikkan duty cycle sebesar 10 (batas atas 255)
-	Menurunkan duty cycle sebesar 10 (batas bawah 0)
PWM:xxx	Mengatur duty cycle langsung (0 sampai 255), misalnya PWM:150

Beberapa konsep pemrograman yang dipakai pada tahap ini: konfigurasi PWM dengan ledcSetup() dan ledcAttachPin(), pembatasan nilai (clamping) agar duty cycle tidak keluar dari 0 sampai 255, pemisahan logika ke dalam fungsi-fungsi kecil (MotorOn, MotorOff, naikkanKecepatan, turunkanKecepatan, setDuty, applyMotor), serta parsing string untuk membedakan perintah satu karakter dan perintah berparameter.

4.1.3 Hasil pengujian

Pada Serial Monitor, perintah kenaikan dan penurunan dijawab dengan pesan status berisi nilai duty cycle terbaru, sehingga proses kendali dapat dipantau. Secara fisik, ketika motor mati hanya LED daya yang menyala, sedangkan ketika motor berputar LED biru indikator ikut menyala.



Gambar 3. Serial Monitor Arduino IDE yang merekam respons kendali kecepatan.



Gambar 4. Kondisi kit iMCLab: motor belum berputar (kiri) dan motor berputar dengan LED biru menyala (kanan).

4.2 Modul 2: Pemrograman iMCLab Menggunakan Python

4.2.1 Tujuan dan pendekatan

Pada modul ini kendali kit dipindahkan dari Serial Monitor ke program Python di komputer. Pustaka pySerial dipakai untuk membuka port serial, mengirim karakter perintah, dan membaca balasan dari ESP32. Pendekatan ini penting karena membuka jalan untuk otomasi, pengambilan data, dan integrasi dengan pustaka Python lain.

4.2.2 Deteksi port otomatis

Nomor port COM dapat berubah setiap kali kabel dicolokkan ulang atau berpindah komputer. Untuk mengatasinya dibuat fungsi deteksi_port_esp32() yang memberi skor pada setiap port yang terdeteksi:

13. **+100** jika pasangan VID:PID cocok dengan chip USB-UART yang umum pada ESP32 (CP210x, CH340, FTDI, atau Espressif).
14. **+50** jika deskripsi, produsen, atau hardware ID memuat kata kunci seperti cp210, ch340, ftdi, silicon labs, atau esp32.
15. **-200** jika port berupa Bluetooth, karena Windows sering menampilkan port Bluetooth virtual yang tidak relevan.

Port dengan skor tertinggi dipilih. Jika tidak ada kandidat tetapi hanya ada satu port non-Bluetooth, port itu dipakai sebagai cadangan (fallback). Pada pengujian, dari tiga port yang terdeteksi (COM11, COM7, COM6), program memilih COM11 (Silicon Labs CP210x, skor 150) dengan benar dan mengabaikan dua port Bluetooth.

4.2.3 Alur program

1. Menampilkan daftar port yang tersedia.
2. Mendeteksi port ESP32 secara otomatis.
3. Membuka koneksi serial pada 115200 baud dan menunggu 2 detik karena ESP32 melakukan reset saat port dibuka.
4. Menjalankan urutan uji: motor ON, tunggu 3 detik, OFF, jeda 1 detik, ON 2 detik, lalu OFF.
5. Masuk mode interaktif (1 = ON, 0 = OFF, q = keluar). Pada versi naik-turun ditambahkan + (naik), - (turun), dan pNNN (set PWM langsung).
6. Blok finally memastikan motor dimatikan dan koneksi ditutup, bahkan ketika program dihentikan paksa dengan Ctrl+C.

4.2.4 Hasil pengujian

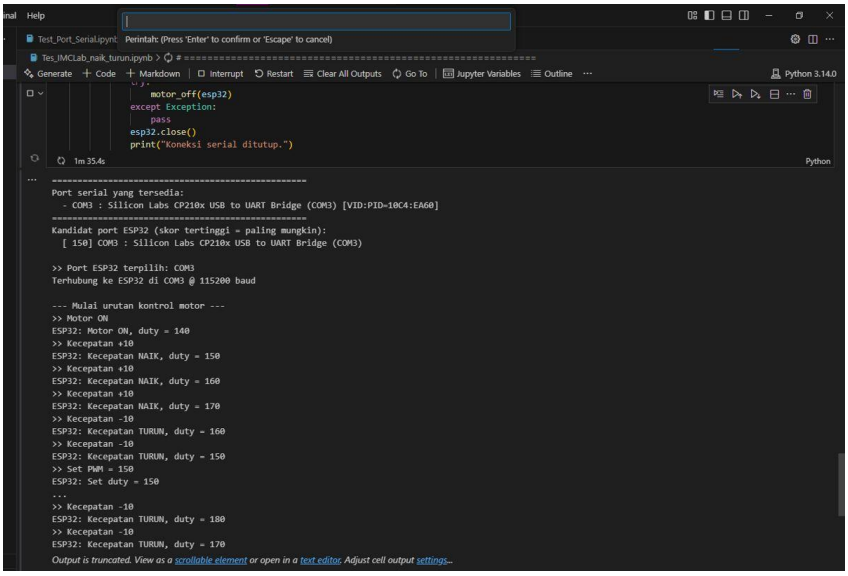
Pada pengujian versi naik-turun, program mengirim urutan perintah ke ESP32 dan menerima balasan yang sesuai. Cuplikan hasilnya ditunjukkan pada Tabel 3.

Tabel 3. Cuplikan urutan perintah Python dan balasan ESP32

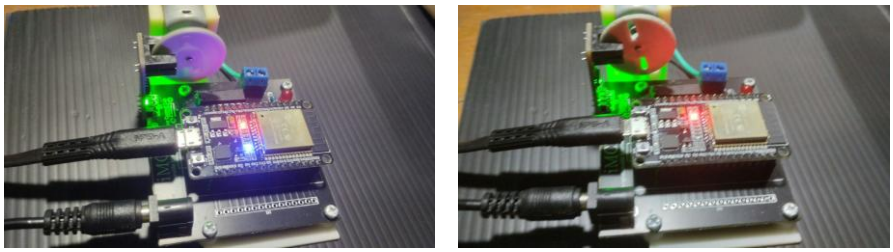
Perintah dari Python	Balasan ESP32
Motor ON	Motor ON, duty = 200
Kecepatan +10 (tiga kali)	Duty 210, 220, lalu 230
Kecepatan -10 (dua kali)	Duty 220, lalu 210

Perintah dari Python	Balasan ESP32
Set PWM = 150	Set duty = 150
Motor OFF	Motor OFF, duty = 0

Hasil ini membuktikan komunikasi dua arah berjalan baik. Satu temuan menarik muncul pada mode interaktif: ketika perintah + dikirim saat motor mati, duty cycle menjadi 10, bukan nilai default. Hal ini terjadi karena state (nilai duty cycle) disimpan di ESP32, bukan di Python. Pelajaran yang diperoleh adalah bahwa sisi komputer perlu mengetahui atau membaca ulang state perangkat, atau urutan perintah harus dirancang dengan memperhatikan state awal.



Gambar 5. Program Python dan log terminal yang menunjukkan program berjalan serta merespons perintah.



Gambar 6. iMCLab merespons kendali dari program Python: motor menyala (kiri) dan motor mati (kanan).

4.3 Modul 3: Pemrograman iMCLab Berbasis Blynk (IoT)

4.3.1 Arsitektur sistem

Pada modul ini ESP32 terhubung ke jaringan Wi-Fi dan ke server Blynk. Dashboard Blynk menyediakan saklar ON/OFF, slider kecepatan, dan tampilan nilai RPM. Setiap widget dipetakan ke virtual pin, seperti pada Tabel 4.

Tabel 4. Pemetaan virtual pin Blynk

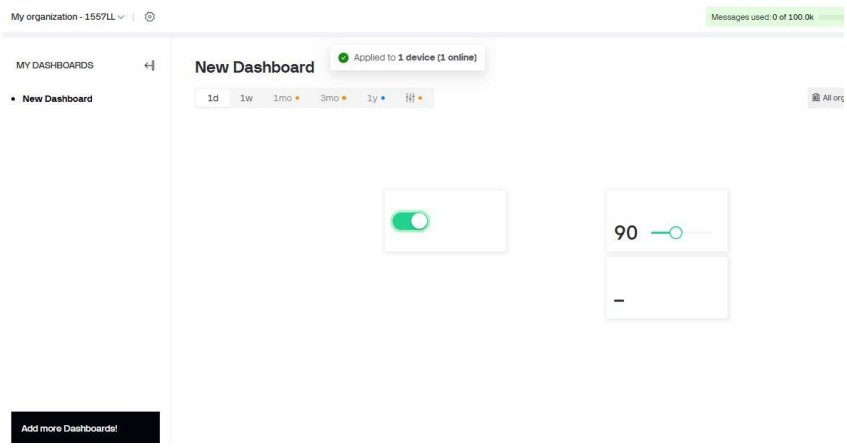
Pin	Widget	Fungsi
V0	Display / gauge	Menampilkan RPM hasil pengukuran sensor
V1	Display	Menampilkan nilai duty cycle (PWM) aktual
V2	Slider	Mengatur duty cycle 0 sampai 255 (BLYNK_WRITE(V2))
V3	Saklar	Motor ON/OFF (BLYNK_WRITE(V3))

4.3.2 Mekanisme penting pada program

16. **Perhitungan RPM dengan interrupt.** Setiap pulsa sensor ditangkap oleh fungsi ISR bertanda IRAM_ATTR dengan debounce 500 µs. Setiap satu detik, jumlah pulsa dibagi jumlah lubang lalu dikalikan 60 untuk memperoleh RPM.
17. **Pengiriman data berkala.** Objek BlynkTimer menjalankan kirimKeBlynk() setiap 1 detik sehingga loop() tidak diblokir oleh delay().
18. **Sinkronisasi dua arah.** Fungsi applyMotor() memperbarui motor sekaligus mengirim nilai terbaru ke V1, V2, dan V3, sehingga tampilan dashboard selalu sama dengan kondisi fisik, apa pun sumber perintahnya (Blynk atau Serial).
19. **Sinkronisasi saat tersambung kembali.** Fungsi BLYNK_CONNECTED() memanggil syncVirtual() agar nilai terakhir di server Blynk diterapkan setelah koneksi pulih.
20. **Kendali ganda.** Perintah serial (1, 0, +, -, PWM:xxx, dan R untuk melihat RPM) tetap berfungsi berdampingan dengan kendali Blynk.

4.3.3 Hasil pengujian

Dashboard Blynk berhasil terhubung ke perangkat (status online). Ketika saklar digeser ke ON, terminal menampilkan log perintah dari Blynk dan motor berputar secara fisik; ketika saklar dimatikan, motor berhenti. Slider kecepatan mengubah duty cycle dan tercatat pada Serial Monitor, misalnya pesan PWM dari Blynk = 90 dan 154. Aplikasi ponsel menampilkan saklar, slider, dan indikator RPM yang sama dengan versi web.



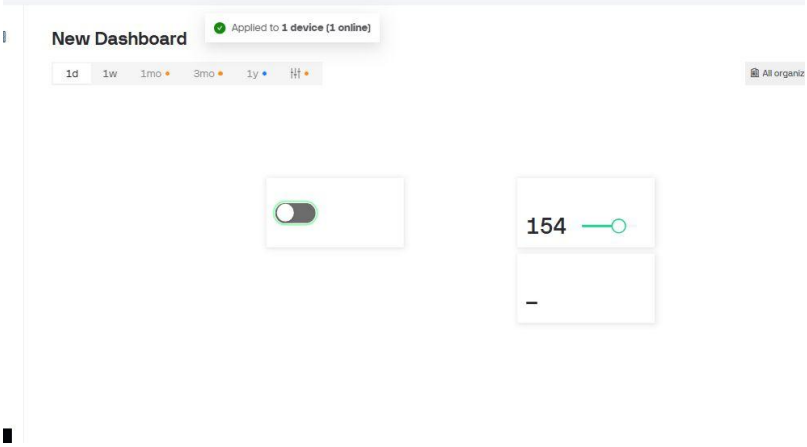
Gambar 7. Dashboard Blynk (web): saklar ON dan slider kecepatan bernilai 90.



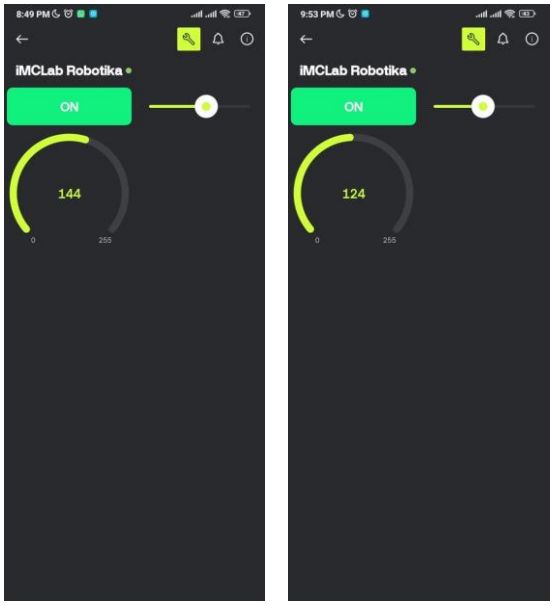
Gambar 8. Serial Monitor yang mencatat perintah dari Blynk (Motor ON/OFF dan PWM dari Blynk).



Gambar 9. Kondisi fisik iMCLab: motor mati (kiri) dan motor menyala lewat Blynk (kanan).



Gambar 10. Dashboard Blynk ketika saklar dimatikan (OFF).



Gambar 11. Tampilan Blynk pada ponsel (aplikasi mobile).

4.4 Modul 4: Pemrograman OTA (Over-The-Air)

4.4.1 Konsep dan implementasi

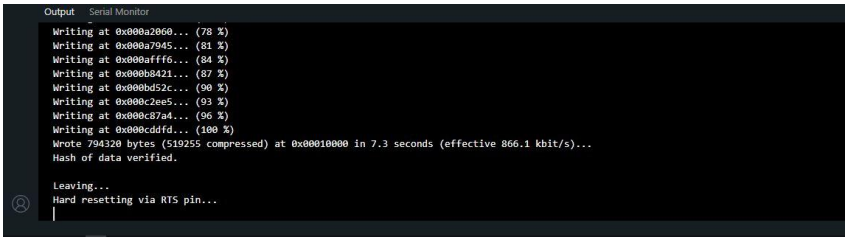
Modul ini menambahkan pustaka ArduinoOTA pada program Blynk. Pada setup() dipanggil ArduinoOTA.setHostname("iMCLab-ESP32") dan ArduinoOTA.begin(), sedangkan pada loop() wajib dipanggil ArduinoOTA.handle() agar ESP32 terus mendengarkan permintaan pembaruan. Tanpa pemanggilan handle() di dalam loop, pembaruan nirkabel tidak akan berjalan.

4.4.2 Prosedur pengujian

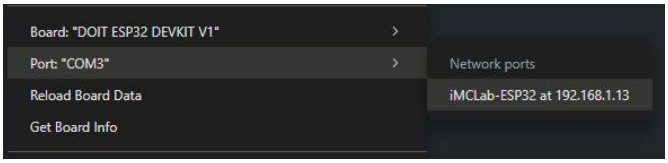
1. Program dasar OTA diunggah satu kali melalui kabel USB (tercatat 794.320 byte tertulis).
2. Kabel USB dilepas dan kit ditenagai langsung oleh adaptor daya.
3. Pada Arduino IDE, menu Port menampilkan bagian Network ports dengan perangkat iMCLab-ESP32 pada alamat IP 192.168.1.13.
4. Port jaringan dipilih, lalu program diunggah ulang; Arduino IDE meminta kata sandi pada kotak dialog Configure and Upload.
5. Firmware dikirim melalui Wi-Fi, dan kemajuannya terlihat pada log (misalnya pada persentase 30 persen).
6. Setelah selesai, kit tetap beroperasi dan motor dapat berputar secara mandiri.

4.4.3 Hasil pengujian

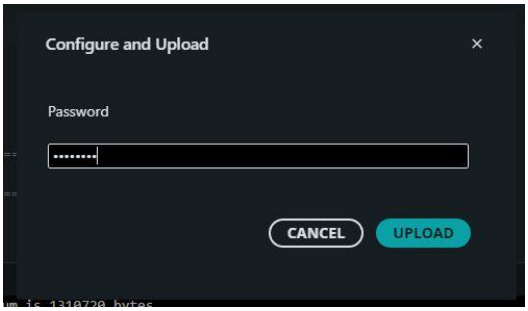
Pembaruan firmware tanpa kabel USB berhasil. Aplikasi Blynk di ponsel tetap dapat memantau perangkat (misalnya menampilkan RPM sekitar 124) meskipun kabel laptop telah dicabut, yang membuktikan perangkat bekerja mandiri melalui jaringan dan adaptor daya.



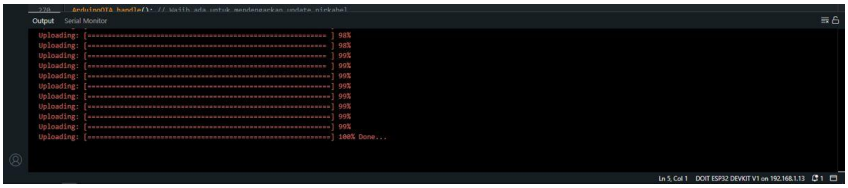
Gambar 12. Log unggah program dasar OTA melalui kabel USB.



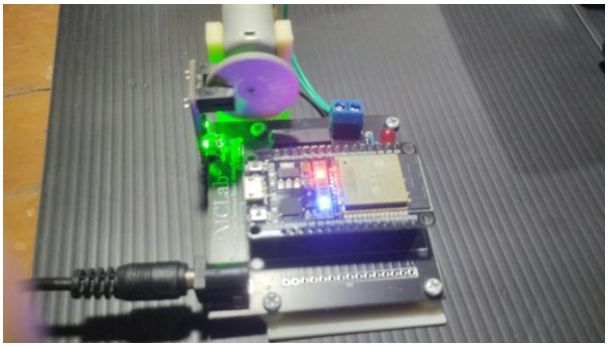
Gambar 13. Network ports pada Arduino IDE yang mendeteksi iMCLab-ESP32 pada alamat IP 192.168.1.13.



Gambar 14. Kotak dialog Configure and Upload untuk memasukkan kata sandi OTA.



Gambar 15. Log proses unggah firmware secara nirkabel melalui Wi-Fi.



Gambar 16. iMCLab dengan motor berputar tanpa kabel USB setelah pembaruan OTA.

BAB V HASIL DAN ANALISIS

5.1 Ringkasan Hasil

Seluruh modul berjalan sesuai tujuan. Tabel 5 merangkum capaian pada setiap modul beserta kompetensi yang dikuasai.

Tabel 5. Ringkasan capaian dan kompetensi

Modul	Yang dikerjakan	Kompetensi yang dikuasai
1. Dasar ESP32 dan PWM	Unggah program, kendali ON/OFF, naik/turun, dan set PWM lewat Serial Monitor	Konfigurasi board dan port, komunikasi serial, PWM (LEDC), parsing perintah, struktur program modular
2. Python	Kendali motor dari Python dengan deteksi port otomatis	pySerial, penanganan eksepsi, deteksi perangkat berbasis VID:PID dan kata kunci, penggunaan try/finally untuk keamanan perangkat
3. Blynk (IoT)	Dashboard ON/OFF, slider PWM, tampilan RPM, aplikasi ponsel	Virtual pin, callback BLYNK_WRITE, BlynkTimer, interrupt sensor, perhitungan RPM, sinkronisasi dua arah
4. OTA	Pembaruan firmware lewat Wi-Fi tanpa USB	ArduinoOTA, hostname dan network port, pengelolaan jaringan lokal, operasi perangkat mandiri

5.2 Kendala dan Solusi

Tabel 6. Kendala yang ditemui dan solusinya

Kendala	Solusi / pelajaran
Nomor port COM berubah-ubah dan ada port Bluetooth virtual	Dibuat deteksi port otomatis berbasis skor (VID:PID, kata kunci, penalti Bluetooth)
ESP32 melakukan reset saat port serial dibuka dari Python	Diberi jeda 2 detik setelah koneksi dibuka sebelum perintah dikirim
Nilai duty cycle tidak sinkron antara Python dan ESP32 (perintah + dari keadaan mati menghasilkan duty 10)	State disimpan di ESP32; urutan perintah dirancang dengan memperhatikan state awal, atau gunakan PWM:xxx untuk nilai pasti
Dashboard Blynk tidak sama dengan kondisi fisik motor	applyMotor() mengirim nilai terbaru ke Blynk setiap ada perubahan dari sumber mana pun
Pembaruan OTA tidak bisa dilakukan tanpa persiapan awal	Kode OTA harus diunggah sekali lewat USB, dan ArduinoOTA.handle() wajib ada di loop()

5.3 Analisis

Rangkaian modul membentuk jalur pembelajaran bertahap. Modul 1 membangun fondasi: bagaimana mikrokontroler menerima perintah dan mengubahnya menjadi sinyal PWM. Modul 2 memperluas kendali ke komputer sehingga proses dapat diotomasi. Modul 3 menghilangkan batasan jarak dengan menghubungkan perangkat ke internet, sekaligus menambahkan umpan balik berupa RPM. Modul 4 menyelesaikan siklus pengembangan dengan memungkinkan pemeliharaan firmware dari jarak jauh.

Dari sisi rancangan program, penggunaan fungsi applyMotor() sebagai satu-satunya titik yang menggerakkan motor dan memperbarui Blynk menjaga konsistensi antarsumber kendali (Serial, Python, dan Blynk). Pemakaian BlynkTimer dan interrupt, sebagai ganti delay(), menjaga loop() tetap responsif sehingga Blynk.run() dan ArduinoOTA.handle() dapat dipanggil terus-menerus.

BAB VI KESIMPULAN DAN SARAN

6.1 Kesimpulan

1. Kecepatan motor DC pada kit iMCLab berhasil diatur menggunakan PWM 8 bit pada frekuensi 30 kHz melalui perintah serial.
2. Kit iMCLab dapat dikendalikan dari Python menggunakan pySerial, dan deteksi port otomatis berbasis skor berhasil memilih port ESP32 yang benar di antara beberapa port.
3. Integrasi dengan Blynk memungkinkan kendali ON/OFF, pengaturan kecepatan, dan pemantauan RPM secara waktu nyata dari web maupun ponsel.
4. OTA terbukti memungkinkan pembaruan firmware melalui Wi-Fi tanpa kabel USB, dengan syarat kode OTA sudah tertanam dan perangkat berada di jaringan yang sama.
5. Secara keseluruhan, praktik ini memberi pemahaman utuh mengenai alur sistem kendali berbasis IoT: aktuator, mikrokontroler, antarmuka komputer, konektivitas awan, dan pemeliharaan jarak jauh.

6.2 Saran dan Rencana Pengembangan

1. Menambahkan pengaman OTA berupa kata sandi pada kode (ArduinoOTA.setPassword) agar hanya pihak berwenang yang dapat memperbarui firmware.
2. Menyimpan kredensial Wi-Fi dan token Blynk pada berkas terpisah yang tidak ikut dibagikan ketika kode dipublikasikan.
3. Menambahkan pembacaan state dari ESP32 ke Python agar kedua sisi selalu sinkron.
4. Mengembangkan pengendali tertutup (misalnya PID) untuk mempertahankan RPM target menggunakan umpan balik sensor.
5. Merekam data RPM terhadap duty cycle untuk menganalisis karakteristik motor dan membuat grafik pada Python.

LAMPIRAN

Lampiran A. Deteksi port otomatis (Python)

```
def deteksi_port_esp32(verbose=True):
    ports = serial.tools.list_ports.comports()
    kandidat = []
    for p in ports:
        teks = f"{p.description} {p.manufacturer} {p.hwid}".lower()
        skor = 0
        if (p.vid, p.pid) in ESP32_VID_PID:
            skor += 100
        if any(kw in teks for kw in ESP32_KEYWORDS):
            skor += 50
        if "bluetooth" in teks:
            skor -= 200
        if skor > 0:
            kandidat.append((skor, p))
    kandidat.sort(key=lambda x: x[0], reverse=True)
    return kandidat[0][1].device if kandidat else None
```

Lampiran B. Penghitungan RPM dengan interrupt (Arduino)

```
void IRAM_ATTR isr() {
  unsigned long now = micros();
  if (now - lastPulseTime > DEBOUNCE_US) {
    rev++;
    lastPulseTime = now;
  }
}

void hitungRPM() {
  if (millis() - old_time >= 1000) {
    detachInterrupt(digitalPinToInterrupt(pin_rpm));
    rpm = ((float)rev / HOLES) * 60.0;
    rev = 0;
    attachInterrupt(digitalPinToInterrupt(pin_rpm), isr, FALLING);
    old_time = millis();
  }
}
```

Lampiran C. Callback Blynk untuk slider dan saklar

```
BLYNK_WRITE(V2) {            // slider PWM
  dutyCycle = constrain(param.asInt(), DUTY_MIN, DUTY_MAX);
  motorEnable = (dutyCycle > 0);
  applyMotor();
}

BLYNK_WRITE(V3) {            // saklar ON/OFF
  motorEnable = param.asInt();
  dutyCycle = motorEnable ? (dutyCycle ? dutyCycle : 200) : 0;
  applyMotor();
}
```

Lampiran D. Inisialisasi dan penanganan OTA

```
// di setup()
Blynk.begin(auth, ssid, pass);
ArduinoOTA.setHostname("iMCLab-ESP32");
ArduinoOTA.begin();
```

```
// di loop()
Blynk.run();
timer.run();
ArduinoOTA.handle();   // wajib agar update nirkabel diterima
```

Catatan: token Blynk, SSID, dan kata sandi Wi-Fi pada kode asli telah disamarkan dan tidak dicantumkan pada laporan ini. Kode lengkap tersedia pada berkas imclab_naik_turun.ino, imclab_Blynk_kelas.ino, OTA.ino, serta notebook Tes_IMCLab_pilih_port_otomatis dan Tes_IMCLab_naik_turun.